1. 튜토리얼 전체 구조와 공통 흐름

SCP Hands-on Labs 튜토리얼 25개(2024.10 기준)를 영역별로 묶은 구성

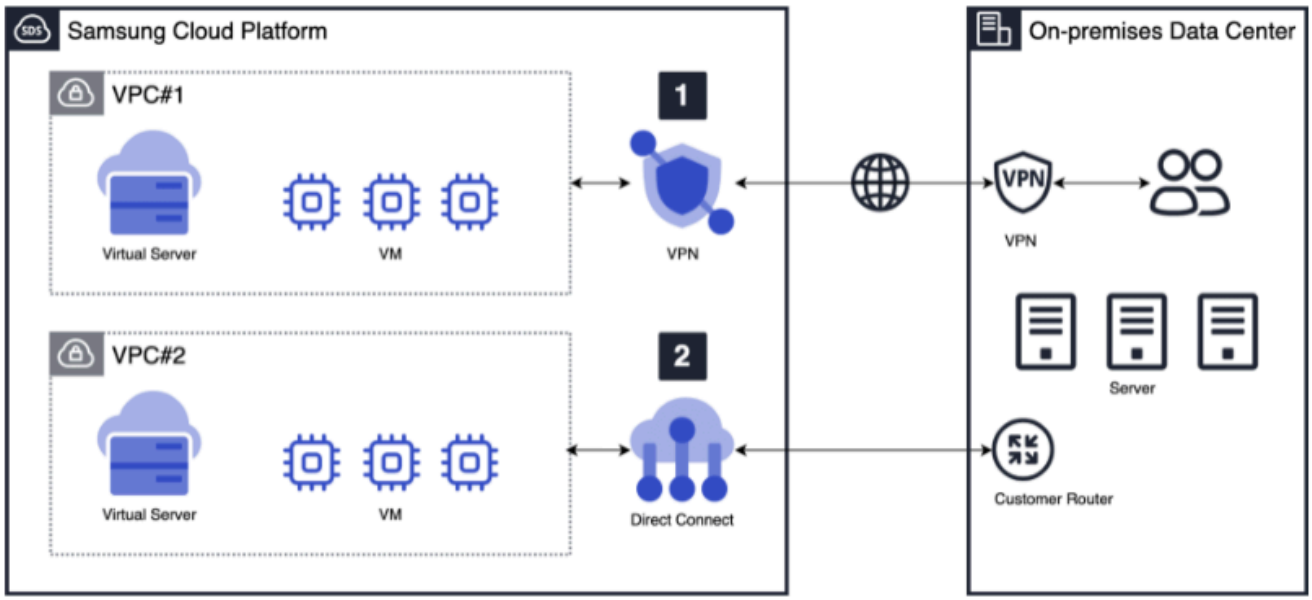
영역	튜토리얼	다루는 개념
Networking	VPC 구성, Public Subnet·인터넷 연결, Security Group, Load Balancer	네트워크 격리, 인터넷 경로, 2단 접근제어, 부하 분산
Compute	Virtual Server, VM 기반 웹 서비스(3-Tier), Auto-Scaling, Scale-In/Out	서버 생성·접속, 이중화, 자동 확장
Storage	File Storage 생성·마운트	공유 스토리지(NAS), NFS/CIFS
Database	PostgreSQL, CacheStore(Redis)	관리형 DB, 백업·복구, HA
Container	K8s 사전 자원, Node CLI, 클러스터, API Server, 외부 인터페이스, Apps	관리형 Kubernetes, 외부 노출 경로
삭제	VPC·SG·LB·AS·FS·DB·K8s 삭제 (9개)	자원 의존성, 정리 순서

모든 실습이 같은 순서를 반복

- Key pair → VPC → Subnet → Internet Gateway(+Firewall) → Security Group → 자원(VM·LB·DB·K8s) → 규칙 추가 → 접속 확인
- 어떤 자원이든 **네트워크와 보안 경계가 먼저 있어야 올라갈 수 있음**
- 삭제 실습이 생성 실습만큼 많음 → 만드는 것보다 **연결 관계를 이해하고 정리하는 것**이 운영의 절반

> **튜토리얼에서 얻는 큰 그림**
>
> - 클라우드 자원은 독립된 상품이 아니라 VPC 안에서 서로 참조하는 의존성 그래프
> - 콘솔 클릭 순서 = 의존성 순서. 삭제는 그 역순
> - 통신이 안 되는 원인 대부분은 서버가 아니라 경로상의 Firewall·Security Group 규칙

2. VPC와 Subnet - 네트워크 격리의 단위



VPC (Virtual Private Cloud)

- 클라우드 안에 논리적으로 격리된 사용자 정의 네트워크. **VPC 없이는 어떤 자원도 생성 불가**
- 관리자가 고객사별 Uplink(인터넷, 그룹망)를 미리 설정 → 사용자는 그 아래에 VPC 생성
- 프로젝트당 VPC 최대 5개, Subnet 최대 25개
- 시스템이 크면 서브시스템 규모·밀접도를 기준으로 VPC를 나눔 (예: 3-Tier 실습의 DMZ용 VPC와 서비스용 VPC 분리)

Subnet 유형

유형	특징	용도
Public	VM에 1:1 공인 NAT IP 설정 가능	Bastion Host, 외부 공개 서버
Private	1:1 공인 NAT IP 설정 불가	Web·WAS·DB 서버
Local	다른 서브넷·외부와 연결 불가, VM 간 직접 통신만	내부 전용 통신

설계 시 고려점

- IP 대역(CIDR)은 생성 시 중복 체크. 실습에서도 "VPC와 CIDR이 맞게 입력되었는지 다시 확인" 강조
- 용도별 서브넷 분리(PUBSUB / PRISUB / DBSUB)가 이후 보안 규칙을 대역 단위로 단순하게 만듦
- Multi Zone VPC는 Zone #1·#2를 모두 포함하고 Subnet도 두 Zone이 같은 대역 사용 → Zone별 서브넷 분리 불필요

3. 인터넷으로 나가는 길 - IGW · NAT · Public IP

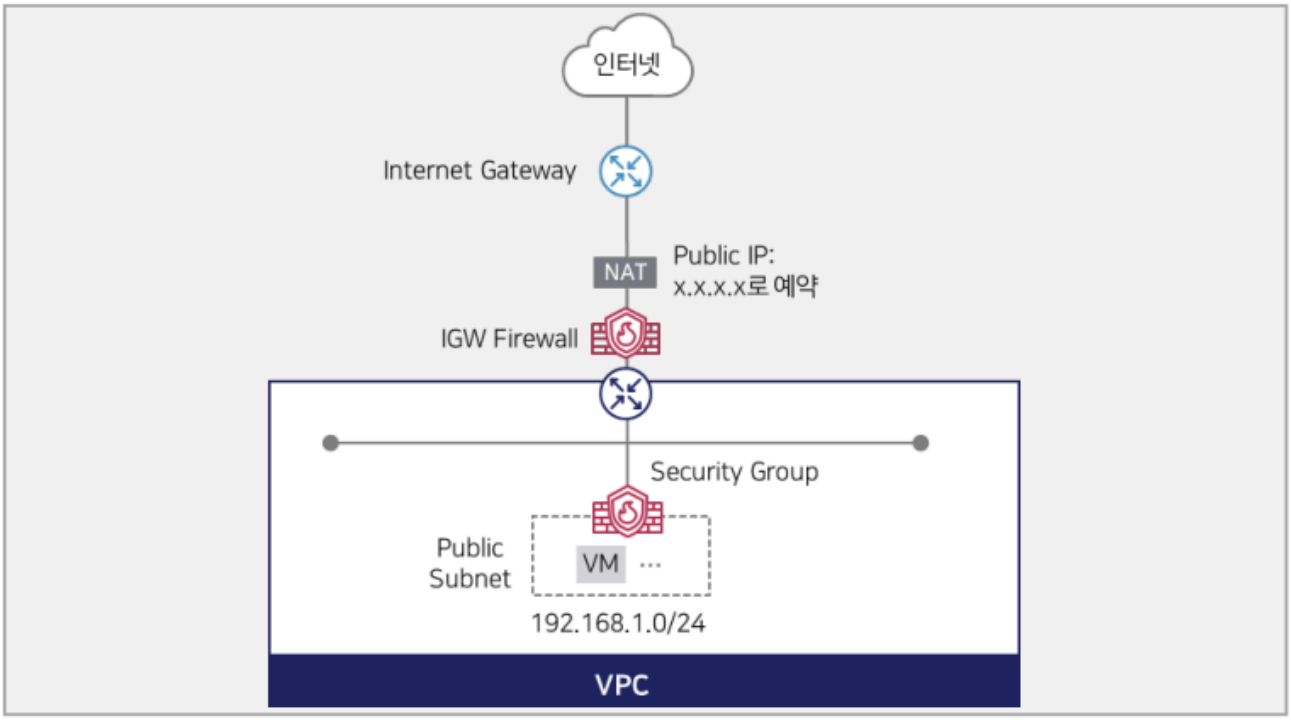


그림 1 Public Subnet 구성하여 인터넷 연결하기 시나리오 구성도

트래픽이 인터넷까지 가는 경로: **VM → Security Group → IGW Firewall → NAT(공인 IP) → Internet Gateway → 인터넷**

구성 요소	역할
Internet Gateway	VPC를 인터넷에 연결하는 업링크. 생성 시 Firewall 사용 체크 → IGW Firewall 자동 생성
NAT IP (1:1)	Public Subnet의 VM 하나에 공인 IP 하나를 매핑. 외부에서 들어오는 접속 가능
NAT Gateway	NAT IP가 없는 VM의 **Outbound만** 대표 공인 IP 1개로 변환. 서브넷 단위 연결
Public IP 예약	특정 공인 IP를 미리 확보해 두고 VM·LB에 지정. 재생성해도 같은 IP 유지

- Private 서버(Web·WAS·DB)는 NAT Gateway로 패키지 업데이트만 나가고, 외부에서 직접 들어올 수 없음
- 외부 → 내부 접속은 LB 서비스의 NAT IP 또는 Bastion Host를 통해서만

> **개념 - NAT**
>
> - 사설 IP와 공인 IP를 서로 바꿔 주는 기술. IPv4 주소 부족 문제에서 출발
> - 1:1 NAT: 양방향 통신 가능 (서버 공개)
> - NAT Gateway(N:1): 내부에서 시작한 Outbound만 가능 (서버 비공개 + 인터넷 사용)
> - 공인 IP를 붙이는 순간 외부 공격 대상이 되므로 "꼭 필요한 서버에만" 부여

4. 2단 접근제어 - Firewall vs Security Group

구분	VPC Firewall (IGW·LB Firewall)	Security Group
위치	VPC 내부와 외부 사이의 경계	자원(VM·DB 등)의 IP 단위
제어 범위	외부 ↔ VPC. VPC 내 Subnet·VM 간 통신은 제어 불가	VPC 내부 자원 간 통신까지 제어
기본 정책	Deny All	Deny All
규칙 종류	Allow·Deny 모두 가능	Allow만 추가
우선순위	있음 (순서 지정)	없음 (Allow 합집합)
규칙 형식	출발지 IP / 목적지 IP / 포트 / 방향	대상 주소(Inbound는 Source, Outbound는 Target) / 포트
한도	프로젝트당 1,000개 (VPC당 200~1,000)	SG당 100개
요금	무료	-

규칙 작성 원칙 (튜토리얼 Tip 정리)

- 외부-내부 규칙의 내부 주소는 **NAT IP가 아닌 내부 IP 대역**으로 입력
- 다수 IP·대역·범위 입력과 우선순위를 조합해 규칙 수를 줄임
- 한 경로에 Firewall이 여러 개 있으면 **모두** 열어야 통신됨 (IGW Firewall + LB Firewall + SG, Direct Connect·Peering 경로의 Firewall 포함)
- LB 뒤의 서버는 SG에 **LB Link IP** Inbound 허용 필요 (LB가 헬스체크·트래픽을 이 IP로 보냄)

통신 장애 점검 순서

- 통신이 안 될 때 서버 프로세스보다 먼저 경로상의 모든 규칙을 따라가며 확인
- 출발지 SG Outbound → (VPC 경계) Firewall Outbound/Inbound → 도착지 SG Inbound
- SG는 상태 기반이라 응답 규칙이 필요 없지만, 요청 방향 규칙이 양쪽(출발 SG Outbound·도착 SG Inbound)에 모두 있어야 함

5. 최소 권한 접근 - Bastion Host 패턴

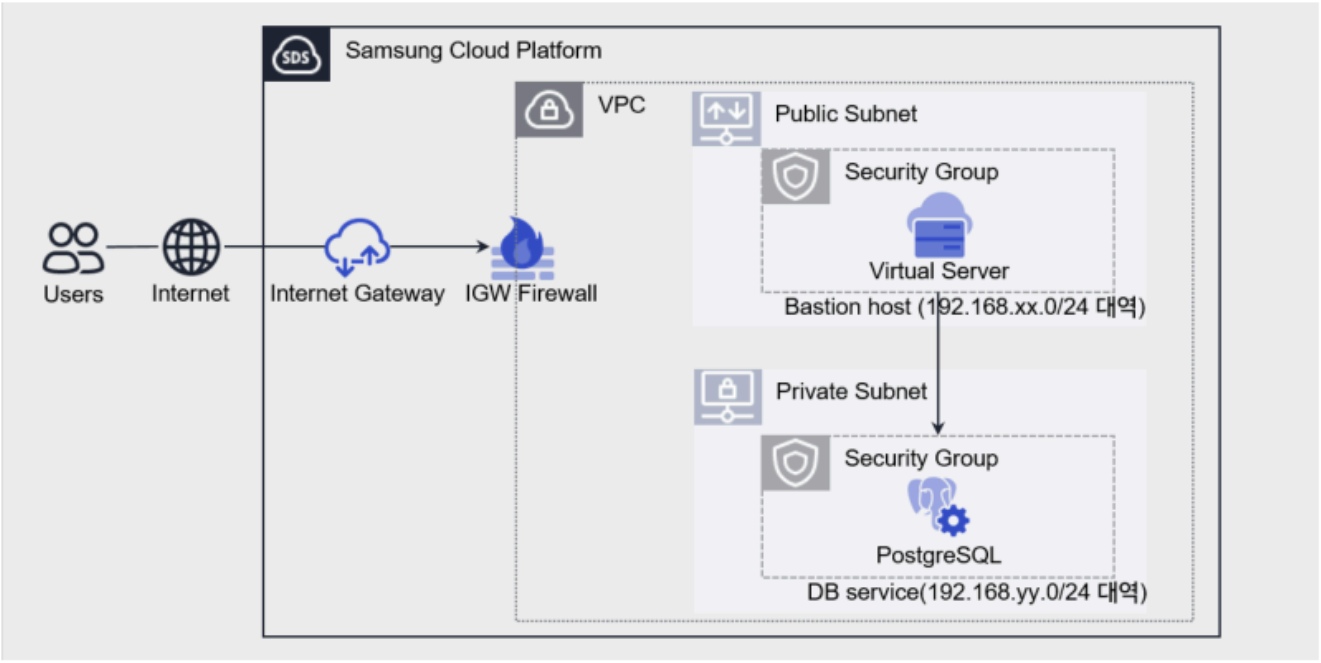


그림 1. PostgreSQL 서비스 생성 시나리오 구성도

구조

- Bastion Host: 제한된 Public Subnet에 둔 관리용 점프 서버. 외부에서 접근 가능한 유일한 관리 통로
- Web·WAS·DB: Private Subnet. 관리자 접속은 Bastion 서브넷 대역에서 오는 22/3389/DB 포트만 허용
- 튜토리얼 정리: **"Bastion은 Public Subnet, Database는 반드시 Private Subnet"**

규칙 설계 예 (3-Tier 실습)

대상	Inbound 허용	Outbound 허용
Bastion SG	내 PC 공인 IP → RDP 3389	인터넷 80·443
Web SG	Bastion 대역 → 관리 포트	WAS 대역·LB 대역 → 9000
WAS SG	Bastion 대역 → 22, Web 대역·LB Link IP → 9000	인터넷 80·443
DB SG	Bastion 대역 → 22·3306, WAS 대역 → 3306	인터넷 80·443

- 계층마다 **바로 앞 계층에서 오는 포트 하나만** 허용 → 한 계층이 뚫려도 다음 계층으로 번지기 어려움
- SG는 같은 보안 규칙을 공유하는 작은 단위(서버 역할별·DB별)로 만드는 것이 정밀한 제어와 관리에 유리
- In/Outbound All Open(Any IP, Any Port)은 자원을 외부 위협에 그대로 노출. Outbound도 마찬가지

6. 서버 접속과 Key pair

Key pair

- 서버 생성 전 Key pair 신청 → **Private Key(.pem)는 최초 1회만 다운로드**
- 분실 시 해당 서버에 키 기반 접속 불가 → 보관이 곧 서버 보안
- Linux: `vmuser` 계정 + Private Key로 SSH 접속 (PuTTY는 .pem → .ppk 변환 필요, MobaXterm은 .pem 직접 사용)
- Windows: 콘솔의 RDP Password 조회에 .pem을 등록해 초기 비밀번호 확인

접속 전제 조건 (모두 충족해야 접속)

1. Public Subnet + NAT IP (또는 Bastion 경유)
2. IGW Firewall Inbound: 내 PC 공인 IP → 22/3389
3. Security Group Inbound: 내 PC 공인 IP → 22/3389
4. 접속하는 장소(사내망)의 방화벽에서 해당 IP·포트 허용

> **내 PC 공인 IP**
> - 규칙의 출발지는 사내 사설 IP가 아닌 외부에서 보이는 공인 IP
> - 장소가 바뀌면 공인 IP도 바뀌므로 접속이 갑자기 안 되면 규칙의 IP부터 확인

7. Virtual Server와 이미지 - 재현 가능한 서버

Virtual Server

- CPU·메모리 등 인프라를 직접 구매하지 않고 필요한 시점에 필요한 만큼 할당받는 가상화 컴퓨팅
- 생성 시 지정: 이미지(OS), 서버 타입(예: s1v1m2 = vCPU 1, Memory 2GB), Block Storage, VPC·Subnet·SG, NAT, AZ, Key pair
- Bastion 용도 서버는 서버 타입·약정 기간·볼륨 암호화·Anti-affinity를 용도에 맞게 선택

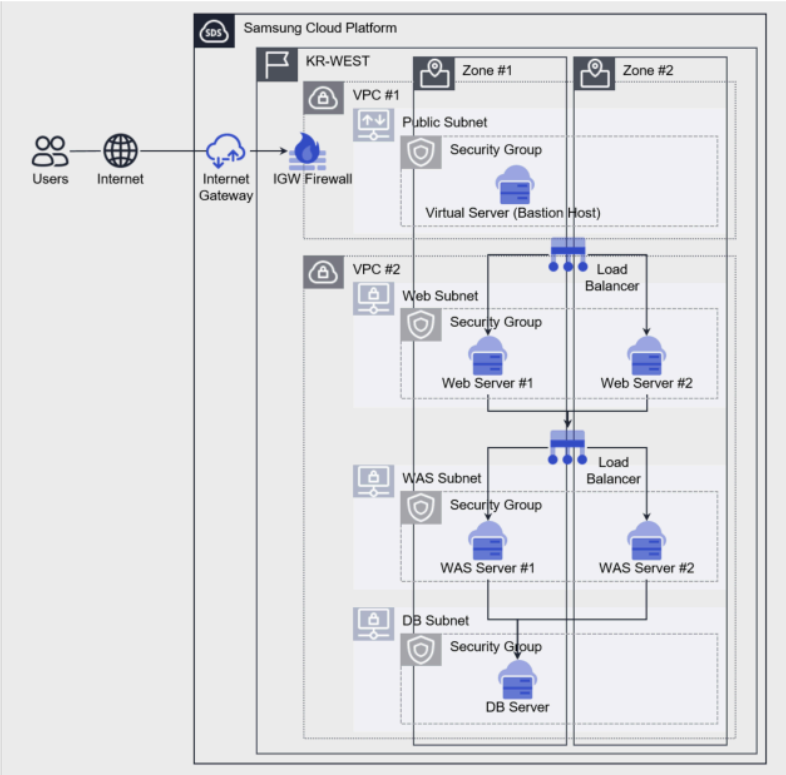
서버를 손으로 만들지 않는 방법 두 가지

방법	동작	튜토리얼 사례
Init script	서버 최초 기동 시 실행되는 스크립트	Web(nginx)·WAS(php) 설치 스크립트를 내려받아 자동 실행
Custom Image	구성이 끝난 서버를 이미지로 저장해 복제	weba1 → `weba1-HA` 이미지 → AZ2에 weba2 생성

- Custom Image는 이중화 서버 추가, Auto-Scaling Launch Configuration에 재사용
- 같은 이미지에서 만든 서버는 설정이 동일 → 이중화 서버 간 차이로 생기는 장애 예방

> **개념 - 불변 인프라(Immutable Infrastructure)**
>
> - 운영 중인 서버를 직접 고치지 않고, 이미지·스크립트를 고쳐 새 서버로 교체하는 방식
> - Auto-Scaling·장애 복구 때 사람 손 없이 같은 서버를 다시 만들 수 있음
> - Init script → Custom Image → Launch Configuration → Terraform(IaC)으로 이어지는 흐름

8. 3-Tier 아키텍처와 Multi-AZ



3-Tier 구성

- Presentation(Web) / Application(WAS) / Database 계층을 각각 다른 서브넷·SG·LB로 분리
- 계층마다 독립된 인프라 → 다른 계층에 영향 없이 업그레이드·확장·축소 가능
- 실습 구성: DMZ VPC(Bastion) + 서비스 VPC(Web·WAS·DB 서브넷), Web·WAS 앞에 각각 LB

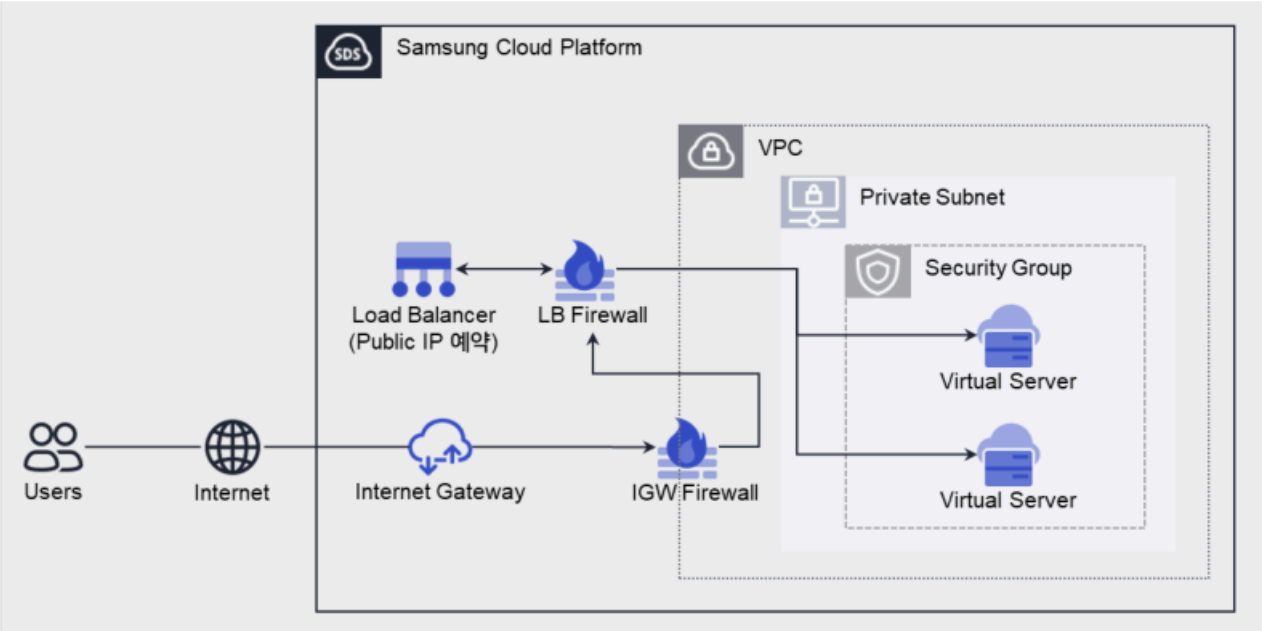
Multi-AZ로 SPOF 제거

- 같은 역할의 서버를 AZ1·AZ2에 하나씩 배치(weba1/weba2, appa1/appa2) → 한 Zone 장애에도 서비스 지속
- SCP Multi Zone 네트워크는 Active-Standby: Active Zone 장애 시 Standby Zone으로 자동 전환, 사용자가 Zone을 고를 필요 없음
- IGW Firewall·LB Firewall 정책은 양 Zone에 동일 적용되고 세션도 동기화
- VPC, LB, Virtual Server, DB, Object/File Storage가 Active-Standby 또는 Active-Active 형태로 가용성 제공

DNS로 서버 대신 이름에 연결

- A 레코드 `www` → Web LB 서비스 IP, `was` → WAS LB 서비스 IP, `db` → DB 서버 IP (TTL 300)
- 애플리케이션 설정에 IP가 아닌 도메인을 넣어 두면 서버 교체·IP 변경 시 레코드만 수정
- 외부 공개는 Public 도메인 A 레코드에 Web LB 서비스의 **NAT IP** 등록

9. Load Balancer 구조



구성 요소 관계: LB 서비스(VIP:Vport) → 서버그룹(멤버:전달 포트), 서비스에 프로파일 연결

구성 요소	역할
Load Balancer	VPC 안에 생성하는 LB 본체. LB 서비스 IP 대역(/27) 지정, LB Firewall 선택
LB 서비스	외부에 보이는 서비스 IP·포트. L4(TCP) 또는 L7(HTTP/HTTPS). NAT IP 연결 시 외부 공개
서버그룹	부하를 받을 서버 목록과 전달 포트, 헬스체크
프로파일	세션 유지 시간 등 애플리케이션 계층 설정

- 분산 방식: Round Robin, Least Connection, IP Hash
- 크기별 LB 서비스 수: Small 20 / Medium 100 / Large 1,000. 동시 세션 수·요금도 크기별로 다름
- 헬스체크 예: TCP, 주기 5초, 대기 5초, 탐지 3회 → 실패 서버는 자동으로 분산 대상에서 빠짐
- 내부 LB(WAS용)는 NAT IP 미사용 → 외부 비공개, Web 계층만 접근
- 동작 확인 방법: VPC 액세스 로그 저장소(Object Storage) 설정 후 LB 액세스 로그 확인

LB를 붙였는데 서비스가 Down일 때

- 서버그룹 서버의 SG에 LB Link IP Inbound 허용이 빠진 경우가 대부분
- LB Firewall 사용 시: 외부 IP → LB 서비스 IP·서비스 포트 규칙 필요
- 외부 경로(IGW·Direct Connect·Peering)에 Firewall이 있으면 그 Firewall도 허용 필요

10. Auto-Scaling - 수요에 맞춰 늘고 줄기

구성 요소

요소	의미
Launch Configuration	확장 시 기동할 서버 템플릿 (OS 이미지·Custom Image, 서버 타입, 디스크, Key pair)
Auto-Scaling Group	서버를 관리하는 논리 그룹. VPC·Subnet·SG·LB 연결
Min / Desired / Max	최소 유지 수 / 목표 수(수동 조정 가능) / 최대 수
정책	사용률 기반 (예: CPU·Memory 임계치) + 쿨다운
스케줄	시간 기반 확장·축소 (예약된 이벤트·업무 시간)
알림	Scale-In/Out 이벤트 발생 시 통지

LB 연계

- ASG에 LB 서비스를 지정하면 서버그룹이 자동 생성·매핑 → 서버가 늘고 줄어도 **서비스 IP는 그대로**
- 새 서버 상태: Creating → Running → LB Attached → ASG In Service

튜토리얼에서 얻는 운영 포인트

- 실습의 Memory ≥ 10% / CPU ≤ 10% 임계치는 이벤트를 강제로 일으키기 위한 테스트 값
- 실제 임계치는 **여러 번의 테스트와 안정화 기간 모니터링**으로 도출
- Scale-Out·Scale-In 조건이 겹치면 늘었다 줄었다 반복(플래핑) → 실습에서도 Out 정책을 지운 뒤 In 정책 추가
- 쿨다운(실습 120초): 확장 직후 지표가 안정될 때까지 다음 조정을 막는 시간

> **개념 - Stateless 서버**
>
> - Auto-Scaling 대상 서버는 언제든 새로 생기고 사라질 수 있어야 함
> - 세션은 CacheStore(Redis), 업로드 파일은 File/Object Storage, 데이터는 DB로 서버 밖에 둠
> - 서버 로컬 디스크에 상태를 두면 Scale-In 때 데이터 유실

11. 스토리지 3종 - 데이터 성격에 맞게 고르기

구분	Block Storage	File Storage	Object Storage
형태	서버에 직접 붙이는 디스크	네트워크 공유 스토리지(NAS)	웹(HTTP) 인터페이스 저장소
접근	서버 1대	여러 서버 동시 마운트	어디서나 API·URL
용도	OS, DB 데이터 디스크	Web/WAS 이중화 파일 공유, 웹 콘텐츠, K8s 볼륨	백업, 로그, 정적 파일, 대용량 보관
튜토리얼 사례	VM 기본 OS 디스크	NFS 마운트, K8s Engine 연결	Firewall 로그, LB 액세스 로그, DB 백업

File Storage

- 프로토콜: NFS(Linux/Unix) / CIFS(Windows, SMB 확장)
- 생성 후 **적용 서버**를 등록해야 접근 가능 (등록 중 Editing → Active)
- 마운트 예: `mount -t nfs -o vers=3,noresvport <IP>:/<경로> /NAS`
- 데이터 저장 암호화 기본 적용
- 신청·관리·해지에 프로젝트 Administrator Group 권한 필요
- Kubernetes Engine 생성 시 **필수 선행 상품**

12. 관리형 DB(DBaaS) - 무엇을 맡기고 무엇이 남는가

PostgreSQL·CacheStore 같은 DBaaS가 자동화해 주는 것

- 설치·프로비저닝, 고가용성 구성, 읽기 복제, 백업·복구, 패치, 파라미터 관리, 감사, 모니터링
- 사용자에게 남는 것: 네트워크 배치(Private Subnet), SG 설계, 접속 도구(pgAdmin 등), 데이터·스키마 관리

가동 관리 동작 차이

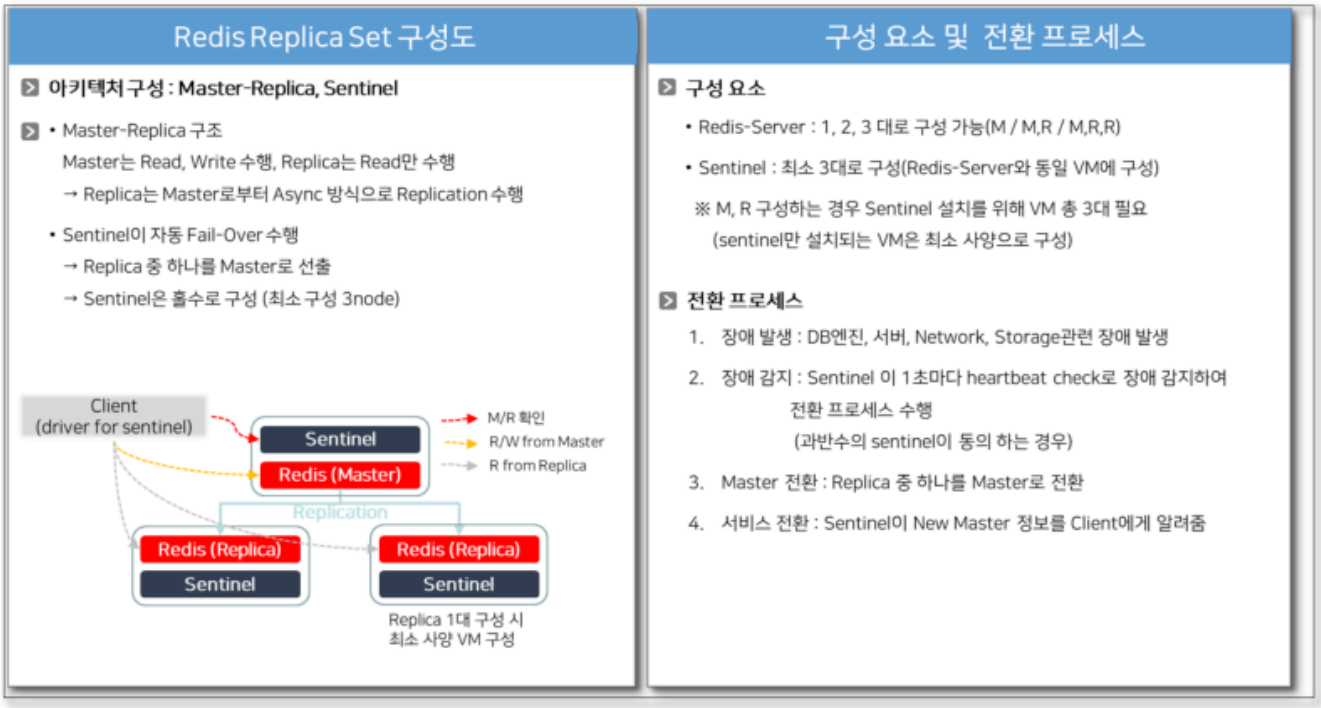
동작	VM	DB 프로세스
시작	기동	기동
중지	종료	종료
재시작	유지	프로세스만 재시작
상품 해지	**VM까지 삭제**	삭제

백업과 복구

- DBMS 자체 백업 기능 사용, 스냅샷 미제공
- 콘솔에서 백업 주기·시간대 지정, 보관 기간 7~35일, 백업 파일은 Object Storage에 저장
- 복구는 백업 시점 기준 **시점 복구**(완전 복구 아님) → 새 VM을 만들고 백업 파일을 Restore
- 복구 VM은 원본 생성 당시 OS·DB 버전으로 설치되고 가동 관리만 제공 (Scale-Up, 백업 설정, 모니터링 미제공)

> **파라미터 변경**
>
> - Dynamic 파라미터: 즉시 반영
> - Static 파라미터: 저장 시 DB Restart 수행 → 운영 중이면 서비스 중단
> - 변경 전 현재 값 조회 필수. 운영 환경은 작업 시간대를 정해 반영

13. CacheStore(Redis) 고가용성



구조

- Master: 읽기·쓰기 / Replica: 읽기 전용, Master 데이터를 비동기 복제 (최대 2개)
- Sentinel: Master 상태를 감시하다 장애 시 Replica 하나를 새 Master로 승격 (자동 Failover)
- Sentinel은 다수결로 장애를 판단하므로 최소 3개(홀수) 필요 → Replica 1개 구성이면 Sentinel 전용 VM 추가

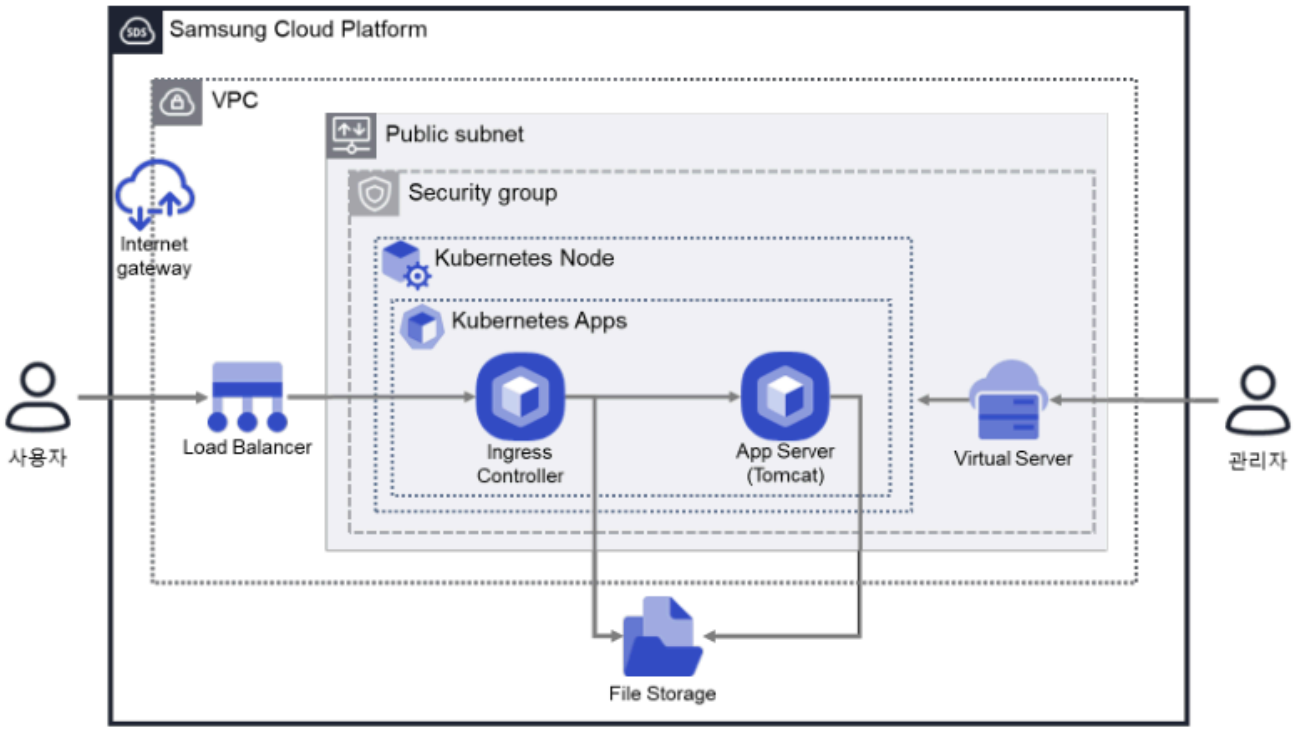
Failover 흐름

1. 장애 발생 (DB 엔진·서버·네트워크·스토리지)
2. Sentinel이 1초 주기 heartbeat로 감지, 과반 동의 시 전환 시작
3. Replica 중 하나를 Master로 승격
4. Client에 새 Master 정보 전달

Replica 수	구성	특징
1	Master + Replica (+ Sentinel VM)	최소 가용성, 읽기 분산, 비용 절감
2	Master + Replica + Replica	더 견고한 가용성, 읽기 분산

- 튜토리얼은 Single로 실습했지만 **운영 환경은 HA 구성 필수**로 안내
- 클라이언트는 Sentinel을 통해 Master를 찾도록 구성해야 Failover 후에도 자동 재연결

14. Kubernetes Engine 구조



관리 영역 구분

구분	내용	관리 주체
Control Plane (제어 영역)	API Server 등 클러스터 두뇌	SCP
Node Pool	Pod가 실제로 뜨는 Worker Node 묶음. 각 노드는 Virtual Server로 조회됨	사용자 (수·타입·자동 확장·자동 복구 선택)

선행 상품: VPC, Security Group, Load Balancer, File Storage → K8s도 결국 VPC 위의 VM·LB·NAS 조합

API Server 접근 설계

- 퍼블릭 엔드포인트 사용 안 함 + 프라이빗 엔드포인트에 **Bastion Host 등록** → 클러스터 관리는 Bastion에서만
- Bastion에 kubectl 설치 후 콘솔의 kubeconfig를 `~/.kube/config`에 저장해야 접속 (없으면 localhost:8080 refused)
- kubectl은 클러스터 버전과 맞추는 것을 권장, 지원 범위는 ±1 마이너 버전
- 제어 영역 로깅 사용 → API 호출 이력 추적

> **개념 - 관리형 Kubernetes**
>
> - Control Plane 설치·이중화·업그레이드를 클라우드가 맡고 사용자는 워크로드와 노드만 관리
> - 노드도 결국 VM이므로 네트워크·SG·비용 관리는 VM과 동일하게 적용
> - Node Pool 단위로 서버 타입을 나누면 워크로드 성격별 배치 가능

15. Kubernetes 외부 노출 경로

트래픽 경로: 사용자 → LB 서비스(80) → 서버그룹(Worker Node:NodePort) → Ingress Controller → Service → Pod(Tomcat)

단계별 개념

- **Kubernetes Apps**: SCP가 제공하는 앱 카탈로그(SCP 3.0 기준 39개). 클러스터·Namespace를 고르고 Ingress Controller, Tomcat 등을 콘솔에서 배포
- **NodePort**: Ingress Controller 최초 배포 시 Service Type. 모든 노드의 같은 포트(예: 32615)로 열림, External IP 없음
- **LB 연결**: 서버그룹 대상 = Node Pool VM 2대 + NodePort, 헬스체크도 NodePort. LB 서비스 80 → NodePort
- **Ingress**: 도메인(Host) 기준으로 요청을 Service에 연결. 앱 배포 시 Service Type을 Ingress로, Domain·Class(nginx) 지정
- Ingress Controller만 있고 규칙이 없으면 404 → 경로는 정상, 연결할 앱이 없다는 뜻
- 실제 도메인이 없으면 DNS 대신 hosts 파일에 External IP·도메인 등록해 확인

튜토리얼에서 얻는 포인트

- K8s 내부 네트워크와 클라우드 LB는 별개 계층 → NodePort가 두 계층을 잇는 접점
- 404·502 등 오류 위치로 어느 계층까지 도달했는지 판단 가능
- Pod 리소스(vCPU 0.2, Memory 0.1)는 노드 여유 안에서만 배치 → Pod 수를 늘리려면 Node Pool 용량부터 확인

16. 로그 · 모니터링 · 알림

튜토리얼 곳곳에 나오는 "기록을 남기는 설정" 정리

대상	설정 위치	저장·확인
IGW·LB Firewall 로그	Firewall 로깅 여부 체크	Object Storage 버킷 지정
Security Group 로그	SG 로깅 여부	-
VPC 액세스 로그 (LB 접속 기록)	VPC 상세 > 로그 저장소	Object Storage에서 로그 파일 다운로드
Auto-Scaling 이벤트	ASG 이벤트 탭, 알림 설정	Scale-In/Out 이력·통지
K8s 제어 영역 로그	클러스터 생성 시 제어 영역 로깅	API Server 호출 기록
DB 백업 이력	DB 상세 > 백업 이력	메타데이터 기반 Web UI
작업 이력	상품 상세 > 작업이력 탭	서버 적용·변경 이력

- 로그는 **사고가 난 뒤에 켜면 이미 늦음** → 로그 저장소(버킷)를 먼저 만들고 생성 시점에 로깅 지정
- 로그 저장소도 VPC 연결 항목 → VPC 삭제 전 해제 대상
- 실습은 비용 때문에 로깅을 껐지만 운영은 Firewall·접속 로그를 남기는 것이 기본

17. 자원 의존성과 삭제 순서

원칙: 생성의 역순. 참조하는 쪽을 먼저 지우고, 참조되는 쪽을 나중에

삭제 대상	먼저 정리할 것
VPC	Subnet, IGW·NAT Gateway, Security Group, VPC Peering·TGW·Direct Connect 연결, 로그 저장소 설정, Auto-Scaling Group, Firewall 규칙 전부
Load Balancer	LB 서비스 → LB 서버그룹 → LB 프로파일 순, 연결된 Firewall 규칙
Auto-Scaling	LB보다 먼저 삭제 (ASG가 쓰는 LB 서비스는 삭제 불가). 해지 시 LB Detach 후 서버 삭제
Launch Configuration	사용하는 ASG가 없을 때만
Kubernetes Engine	Kubernetes Apps → Node Pool(노드 수 0) → 클러스터(Control Plane)
Security Group	적용 상품 수 0 (자원 삭제 또는 다른 SG로 교체)
File Storage	적용 서버 0 (서버 삭제 또는 연결 해제)
Database	이름 재입력 확인. 연결 Block Storage는 함께 삭제, **Replica는 별도 삭제**

함께 지워지지 않는 자원

- K8s 삭제 후에도 **File Storage는 남음** (적용 서버만 해제됨)
- DB 삭제 후에도 **백업 버킷(Object Storage)은 남음**
- 예약한 Public IP, Custom Image, Key pair도 별도 정리

> **남은 자원 = 비용 + 보안 구멍**
>
> - 정리 누락 자원은 계속 과금되고, 열린 규칙·공인 IP는 공격 경로로 남음
> - 정기적으로 태그(예: Key `SCP`, Value 사용자) 기준으로 소유자 없는 자원 점검

18. 비용 관점에서 본 튜토리얼

튜토리얼에 나온 과금 포인트

- VPC 구성 자체보다 **Internet Outbound 트래픽, Public IP, NAT Gateway, VPC Peering** 추가 시 과금
- Firewall은 무료 → 비용 때문에 Firewall을 생략할 이유 없음
- LB는 크기(Small·Medium·Large)별 요금 → 필요한 서비스 수·세션 수 기준으로 선택
- K8s 클러스터는 신청 화면에서 금액 확인. Node Pool VM이 별도 과금되는 구조
- DB 백업 보관 기간(7~35일)이 길수록 Object Storage 사용량 증가

비용을 줄이는 설계 습관

- 공인 IP는 Bastion·외부 LB 등 꼭 필요한 곳에만. 나머지는 NAT Gateway 공유
- Auto-Scaling Min을 평시 부하 기준으로, Max로 비용 상한 설정
- 서버 타입은 작게 시작해 모니터링 후 조정
- 실습·개발 자원은 사용 후 즉시 삭제 (17장 순서 참고)
- 태그로 프로젝트·담당자를 붙여 비용 출처 추적

19. 클라우드 이용·운영 시 반드시 알아야 할 것

영역	체크 항목
설계	VPC·Subnet CIDR을 미리 설계하고 중복 방지. 용도(Public/Private/DB)별 서브넷 분리
설계	VPC 5개·Subnet 25개·SG 규칙 100개·LB 크기별 서비스 수 등 한도를 설계 단계에서 반영
보안	Firewall·SG 모두 기본 Deny All. 필요한 IP·포트만 Allow, Any Open 금지
보안	DB·애플리케이션은 Private Subnet, 관리 접속은 Bastion 경유
보안	Private Key는 1회 다운로드 → 안전하게 보관. 분실은 접속 불가로 직결
보안	K8s API Server는 프라이빗 엔드포인트 + Bastion
가용성	서버는 최소 2대를 서로 다른 AZ에, 앞단에 LB + 헬스체크
가용성	DB·Redis는 운영 환경에서 HA(Replica) 구성
확장	서버는 Stateless로, 상태는 DB·Redis·File/Object Storage로 분리
확장	Scaling 임계치는 모니터링으로 도출, In/Out 조건 겹침 방지
자동화	Init script·Custom Image·Launch Configuration으로 서버를 재현 가능하게
운영	로그 저장소를 먼저 만들고 Firewall·접속 로그 활성화
운영	Static 파라미터 변경 = DB 재시작. 작업 시간 계획
운영	백업은 보관 기간·복구 방식(시점 복구, 새 VM)까지 확인
정리	삭제는 생성의 역순. File Storage·백업 버킷·Replica·Public IP 잔존 확인

20. 튜토리얼에서 얻은 인사이트

1) 클라우드는 "상품 목록"이 아니라 "연결 관계"

- VM 하나를 띄워도 VPC·Subnet·IGW·Firewall·SG·NAT·Key pair가 함께 움직임
- 삭제 실습에서 연결 고리가 끊기지 않으면 아무것도 지워지지 않는 경험 → 구성도를 의존성 그래프로 보는 시각

2) 보안은 겹겹이, 기본값은 닫힘

- 경계(Firewall) + 자원(SG) + 위치(Private Subnet) + 경로(Bastion) 4중 구조
- 기본 Deny All 덕분에 "열지 않은 것은 막혀 있음" → 장애 분석도 "무엇을 안 열었나"에서 시작

3) 가용성은 설계로 만드는 것

- Multi-AZ 배치, LB 헬스체크, Redis Sentinel, Auto-Scaling 모두 "한 대가 죽어도 된다"는 전제
- 서버를 소중히 다루기보다 언제든 교체 가능하게 만드는 쪽이 더 안정적

4) 관리형 서비스는 책임 분담

- DBaaS·Kubernetes Engine은 설치·이중화·백업을 맡아 주지만 네트워크 배치, 접근제어, 파라미터 영향, 데이터 보존은 사용자 몫
- 무엇을 맡겼는지(Control Plane, 백업 자동화)와 무엇이 남았는지(Node Pool, SG, 복구 절차) 구분 필요

5) 콘솔 실습 다음 단계는 코드

- 같은 VPC→Subnet→SG→VM 순서를 25번 반복하며 느낀 수작업의 한계
- 반복 구성은 Terraform 같은 IaC로 옮겨야 재현성·검토·정리가 쉬워짐 (Hands-on Labs-2의 Terraform Advanced로 이어지는 흐름)